T1

注意到，只要确定了雷在哪些位置，那么整个矩阵都确定了，因此矩阵计数转为对于雷的集合计数。

而后者显然为 2^{nm}，这就是答案。

T2

我们首先来说明一个结论，任意一个边双都可以被定向成强连通分量。

定向方案是简单的，任选一个点开始 DFS，所有树边定向成父亲到儿子，所有非树边定向成深度大的点指向深度小的点。考虑边双的定义，如果存在一个点 x，DFS 上 x 子树中没有能到子树外的非树边，那么 x 的父亲到 x 的这条边就是割边了。因此，边双 DFS 树任意一个子树中都存在到子树外的非树边。

于是对于我们的定向方案，任意一个点都可以不断重复：先走到子树中某个点，再跳出子树的过程，一直到走到根。而根显然可以到达所有点，因此这样定向保证了任意两个点互相可达，这是一个 SCC。

于是定向后，边双中的任意两个点都互相可达，我们可以把边双缩点，这样就规约成树的问题了。

而对于树定向使得点对可达，这一问题是简单的，只需要每条树边只在至多一种方向上被经过，树上前缀和即可。

因此瓶颈在于树上 lca，用你喜欢的方式求 lca 即可，复杂度依实现做到线性或者线性对数。

T3

一道很有教育意义的题目。

不过好像题解疑似写得太形式化了，谢罪/ll

以下是一些考察计数基本功的转化：

一个点 x 的权值是 $(t+deg_x)$，设保留下来的边集为 E'，那么可以把权值改写为 $t+\sum_{(x,y)\in E'}1$。

而总贡献是每个点的乘积，我们将乘积拆开，变成每个点恰好选择其中一项：

1. 不钦定任何一条边在 E' 里，权值为 t
2. 钦定一条边在 E' 里，权值为 1

最后乘上这种方案的概率，也就是对于每一条钦定在 E' 里的边，乘上其保留的概率 p。

把所有选择得到的结果求和，就是答案。

暴力枚举每个点的选择，你得到了一个 $O(\prod_{x=1}^{n}(deg_x+1))$ 的做法。

接着很容易想到，我们可以在某个点钦定一条邻边选择的时候，就乘上其对应的概率。但是这样有一点问题，对于一条边 (u,v,p) 来说，如果 u 和 v 同时选择了它就会产生 p^2 的权，而本来应该产生 p 的权。

所以我们可以“广义容斥”，钦定一些边被其连接的两个点同时选择，并且认为每条这样的边造成 $p-p^2$ 的权。除此之外，所有没有被钦定选择边的点 u，贡献直接就是任选的权值和，也就是 $w_u=t+\sum_{(u,v)\in E}p_{(u,v)}$。

所以此时我们转为钦定一个边集 S，要求边集中任意两条边不共点。一条被选择的边贡献 $p-p^2$，一个不在 S 中的点 x 贡献 w_x，那么钦定 S 得到的权值就是所有的贡献乘积，最后对于所有 S 的权值求和就是答案。

一次考虑每条边是否在 S 里，状压每个点是否已经被钦定过，你就得到了一个 $O(2^n m)$ 做法。

接着来，题目中的性质明示我们按照 k 将 $[0,n-1]$ 分块，那么连边只会在块内，或者相邻两个块之间。并且有一个很特殊的性质是，第 i 个块中每个点，至多往 2 个 $i+1$ 块中的点连边。

先不考虑这个特殊的性质，直接逐块考虑，那么显然只需要考虑相邻两个块直接的点是否已经被选择了，所以直接状压相邻两个块是 $O(2^{2k}m)$。

再考虑这个特殊的性质，我们先处理一个块内部的连边，再处理其往下一个块的连边。设 $f_{i,S}$ 表示当前处理到第 i 个块，并且块内点选择情况是 S 的权值和，先考虑所有块内的边选还是不选，再考虑每个点是否不钦定邻边在 S 内，直接产生 w_x 的贡献。如果最后剩下来 V 集合中的点还没有选择，那么可以知道，V 中的每个点都必然和 $i+1$ 块中的某个点连边。

因为对于每个点来说，这两的连边至多只有两条，所以可以 $O(2^{|V|})$ 枚举是与下个块中的哪个点有连边，那么就可以把 $f_{i,U/V}$ 转移到 $f_{i+1,T}$，其中 T 是 V 中所有点在第 $i+1$ 块中选择的所有点的并集，这里也要求任意两个 V 中点选择连接的点不同。

分析一下复杂度，大概是 $O(n3^k+2^k m)$ 的。特别的，sub4 的复杂度也是正确的 $O(2^k(n+m))$，因为每个点事实上只有 1 种选择。

如果将枚举改成搜索剪枝，时刻要求每个点选择连接的点在之前不能被选过，可以感受到 3^k 跑不满。我的粗略实现洛谷数据可以拿 83pts，看上去 breakplus 的实现可以直接冲过去。

然后是一些有新意的想法了：

在反复思考这个题的过程中，我总是想到一个名词“轮廓线 DP”，就是在 DP 的过程中，只状压那些需要记录的点。但是在这个题中，每次考虑完第 i 个块中的一个点，都会使得我们需要额外记录第 $i+1$ 个块中的两个点，看上去根本没有办法减少需要记录的点。

但是！我们真的没有办法减少需要记录的点吗？考虑更形式化的，用图论语言描述一下，到底哪些点是需要记录的。

对于一个第 i 个块中的点 u，如果其与第 $i+1$ 个块没有连边，那么什么也不做；

对于一个第 i 个块中的点 u，如果其与第 $i+1$ 个块有连边 (u,x)，那么在新图上连边 (x,x)。

对于一个第 i 个块中的点 u，如果其与第 $i+1$ 个块有连边 $(u,x),(u,y)$，那么在新图上连边 (x,y)。

注意到，我们并不需要按照顺序依次考虑当前块中的每个点，考虑的顺序是任意的。如果每次加入一个点得到的点集分别是 $S_1,S_2,\dots,S_k$，可以发现我们需要记录的点的数量其实是：

$$\max_{i=1}^{k} k-S_i+T_i$$

其中 T_i 是 S_i 中点在新图中对应的边的导出子图点集。

而事实上，存在一种加点顺序使得这个值恒 $\le k+1$！

考虑对于新图中的每个连通块，我们按照 DFS 的顺序依次加入每条树边，再加入所有的非树边，那么对于一个连通块就始终能够保证要记录的点数不超过边数 $+1$；

而对于多个连通块，我们将其按照点数减边数的顺序从小到大排序，再依次考虑每个连通块，可以发现整个过程中都有点数不超过边数 $+1$！

也就是意味着，按照这样的顺序考虑，我们要记录的点始终不超过 $k+1$！

因此此时再上轮廓线 DP 进行两个块直接的状态转移，复杂度就变成了 $O(2^{k+1}n+2^k m)$。

有一些细节，想清楚之后再写就能够轻松通过了。

T4

Intersting，猫树分治还能这么玩，赛时回滚莫队硬是凭着 pq 的超小常数 $O(n\sqrt{n}\log n)$ 过了 2×10^5。

我们考虑对于区间进行分治，在每个询问区间第一次跨过端点的时刻处理，只需要计算出 $[ql,mid],[mid+1,qr]$ 拼成区间的答案。

首先处理两边内部的答案，$[mid+1,qr]$ 是好做的，考虑从左到右扫，用一个堆维护当前所有左括号的最大值，遇到左括号就 push，右括号就 pop。然后考虑 $[l,mid]$ 这一段，我们只能从右往左扫了，因此考虑反悔贪心，用一个堆维护已经匹配了的左括号最小值，如果替换能够使得答案更大那么就替换，这样也就处理出了贡献。

然后考虑跨端点的匹配，不难发现一定是左区间没有匹配的左括号前 k 大，和右区间已经匹配了的左括号前 k 小进行操作，操作就是撤销右边匹配，改成左边匹配。不难发现只需要二分出最大的 k 满足左边第 k 大大于右边第 k 小即可，我们考虑用主席树维护 $[ql,mid]$ 和 $[mid+1,qr]$ 时刻的集合，查询时直接外层二分 k，内层主席树上查询，由于每个询问只会被处理一次所以是 $O(q\log^2 n)$ 的，分治时建主席树也是 $O(n\log^2 n)$ 的，总复杂度两只 $\log$。